

OpenMP

HIGH PERFORMANCE AND CLOUD COMPUTING

16 April 2026

Indice

0.1	OpenMP: Shared-Memory Parallelism in High Performance Computing	2
0.1.1	1. Processes and Threads: Foundational Concepts	2
0.1.2	2. Introduction to OpenMP	3
0.1.3	3. The Fork-Join Execution Model	4
0.1.4	4. The OpenMP Toolbox	5
0.1.5	5. Static and Dynamic Extent	6
0.1.6	6. Parallel Regions	7
0.1.7	7. The Thread Factory: Stack Layout and Memory Organization	8
0.1.8	8. Controlling the Number of Threads	9
0.1.9	9. Specializing Execution Within a Parallel Region	10
0.1.10	10. Shared and Private Variables: The Scoping Rules	13
0.1.11	11. Nested Parallelism	16
0.1.12	12. Parallel Loops	17
0.1.13	13. Data Races and Their Anatomy	19
0.1.14	14. False Sharing	20
0.1.15	15. Solving the Reduction Problem	21
0.1.16	16. Performance Case Study: Monte Carlo Estimation of	23
0.1.17	17. NUMA Awareness and Thread Affinity	24
0.1.18	18. Memory Allocation on NUMA Systems	26
0.1.19	19. The OpenMP Memory Model	28
0.1.20	20. Internal Control Variables (ICVs)	31
0.1.21	21. Summary of Practical Guidelines	32

0.1 OpenMP: Shared-Memory Parallelism in High Performance Computing

0.1.1 1. Processes and Threads: Foundational Concepts

0.1.1.1 1.1 What Is a Process? A **process** is an independent sequence of instructions together with the ensemble of resources the operating system has allocated for their execution. A running program requires far more than its binary code (the list of operations to be executed): it needs access to a protected memory space, file descriptors, network sockets, and other system resources. The operating system bundles all of these into a single entity — the process — identified by a unique **PID** (Process ID).

The virtual address space of a process is organized into well-defined segments:

- **Code segment:** the executable instructions (read-only, shared across instances of the same program).
- **Data segments:** global and static variables (BSS for uninitialized, data segment for initialized).
- **Heap:** dynamically allocated memory, growing upward via `malloc/mmap`.
- **Stack:** local variables, function call frames, growing downward from high addresses.

Multiple instances of the same program can coexist as different, independent processes. Each has its own virtual memory map, and one process cannot directly access the memory of another.

0.1.1.2 1.2 What Is a Thread? A **thread** is an independent instance of code execution within a process. There may be from one to many threads within the same process. Unlike processes, threads within the same process share:

- The same code segment
- The same memory address space (heap and global data)
- The same system resources (file descriptors, network connections)

However, each thread possesses its own:

- **Instruction pointer (IP):** allowing it to execute different parts of the code simultaneously
- **Stack pointer (SP) and base pointer (BP):** defining its own private stack frame
- **Stack:** where thread-local variables reside

The fact that threads share the heap is what makes inter-thread communication inexpensive — and also what makes it dangerous, since concurrent access to shared data can lead to **race conditions**.

A thread can run on the same computational unit (core) as its parent process or on a different one. A computational unit today typically corresponds to a **core**, either within the same CPU socket on which the parent runs or within a sibling socket in the same NUMA region.

Key practical observation: spawning threads inside a process is significantly less costly than creating new processes, because threads do not require duplication of the entire address space.

0.1.1.3 1.3 Threads vs. Processes: A Summary

Property	Process	Thread
Address space	Independent, isolated	Shared with parent and siblings
Communication	Explicit (pipes, sockets, messages)	Implicit (shared memory)
Creation cost	High (full address space duplication)	Low (only stack + registers)
Failure isolation	Complete (one crash → others crash)	None (one crash → all crash)
Typical parallel model	MPI (distributed memory)	OpenMP (shared memory)

0.1.2 2. Introduction to OpenMP

0.1.2.1 2.1 What Is OpenMP? **OpenMP** (*Open specifications for MultiProcessing*) is a standard API for shared-memory parallel programming. It enables the development of multi-threaded programs with standardized behavior through a set of **compiler directives** inserted directly into the source code:

- **Pragmas** (`#pragma omp ...`) in C/C++
- Specially formatted comments in Fortran

OpenMP supports both **fine-grain** and **coarse-grain** parallelism, ranging from loop-level parallelization to explicit work assignment to individual threads.

0.1.2.2 2.2 Advantages of the Directive-Based Approach

1. **Abstraction:** hardware-specific details and subtleties of low-level threading libraries (e.g., `pthread`) are hidden. The programmer can focus on data flow and workflow design.
2. **Efficiency:** the learning curve to achieve reasonable parallel performance is much shallower than with `pthread`. The result-to-effort ratio is highly favorable.
3. **Incremental approach:** there is no need to rewrite an entire codebase. One can begin by parallelizing the most compute-intensive sections, guided by profiling results.
4. **Portability:** the compiler handles platform-specific details. The programmer still must design algorithms that adapt to different topologies, but the directive syntax remains uniform.
5. **One source:** through conditional compilation, serial and parallel versions can coexist in the same source file.

0.1.2.3 2.3 OpenMP vs. MPI OpenMP and MPI represent two fundamentally different paradigms for parallel computing:

Shared-Memory Model (OpenMP): A single process spawns a number of threads. All threads share a unique memory address space. Communication between threads is implicit — they read from and write to the same memory locations.

Distributed-Memory Model (MPI): N independent processes are created, each with its own copy of the code and its own memory space. A process **cannot** directly access the memory space of another process. Communication occurs exclusively through **explicit message passing**.

Hybrid Model (MPI + OpenMP): In practice, many HPC applications combine both paradigms. N MPI processes are created, each of which may internally spawn multiple OpenMP threads. Inter-process communication uses MPI messages, while intra-process parallelism leverages shared memory via OpenMP. This hybrid approach can exploit both multi-node distribution and intra-node shared memory efficiently.

Note: MPI 3.0 introduced features that blur this boundary, including shared-memory windows (allowing shared-like accesses among tasks on the same node) and **Remote Memory Access (RMA)** for direct memory access to the address space of other MPI tasks.

0.1.3 3. The Fork-Join Execution Model

0.1.3.1 3.1 Model Overview OpenMP employs a **fork-join** model of execution:

1. The program begins with a single **master thread** executing serial code.
2. When the master thread encounters a parallel directive, it **forks** a team of worker threads.
3. The master thread joins the new pool as **thread 0**.
4. All threads in the team execute the parallel region concurrently.
5. At the end of the parallel region, all threads **join** — an implicit synchronization barrier ensures all threads have completed before the master thread resumes serial execution.

This cycle of fork and join can repeat throughout the program's lifetime, with serial sections interspersed between parallel regions.

0.1.3.2 3.2 Key Properties of the Model

- Threads access and modify **shared memory regions**; explicit or implicit synchronization protects against race conditions.
- There is **no concept of explicit message passing** — all communication occurs through shared variables.
- **Loop-carried dependencies** hamper any parallel speedup.
- **Shared-variable attributes** are vital to reduce or avoid race conditions and the need for synchronization.
- Each thread performs its portion of parallel work in a **separate stack** that is not visible to other threads or outside the parallel region.
- **Nested parallelism** is explicitly permitted.
- The number of threads can be **dynamically changed** before a parallel region.

0.1.3.3 3.3 Parallel Region Overhead The OpenMP runtime does not necessarily create and destroy threads at every parallel region. For efficiency, the thread pool is typically created once (at the first parallel region or at program start), and threads are put to sleep outside parallel regions. The cost of entering a parallel region is therefore not the cost of spawning POSIX threads from scratch; it is the cost of **waking sleeping threads**, setting up the team, and distributing work. On modern systems, this overhead is of the order of ~ 10 s — small but not negligible, especially if parallel regions are opened inside tight loops.

0.1.4 4. The OpenMP Toolbox

OpenMP consists of three components:

0.1.4.1 4.1 Compiler Directives Directives provide instructions to the compiler about how to manage thread creation, work distribution, data sharing, and synchronization.

The general syntax in C/C++ is:

```
#pragma omp <directive> [<clause> [, <clause> [...]]]
{
    statement;
    ...
}
```

If the code is compiled without OpenMP support (i.e., without `-fopenmp`), the `#pragma` is silently ignored — a design feature ensuring that a well-written OpenMP program compiles and runs correctly as a serial program.

In C++ (OpenMP 5.1), an attribute syntax is also available:

```
[[ omp :: <directive> [<clause> ...] ]]
```

0.1.4.2 4.2 Runtime Library Functions The OpenMP runtime library provides functions for querying and controlling the parallel environment:

Function	Description
<code>omp_get_num_threads()</code>	Number of threads in the current team
<code>omp_get_thread_num()</code>	ID of the calling thread (range $[0, n_{\text{threads}} - 1]$)
<code>omp_set_num_threads(n)</code>	Set default team size from this point onward
<code>omp_get_max_threads()</code>	Upper bound for the next team size
<code>omp_get_wtime()</code>	Wall-clock time in seconds (not synchronized across threads)

Important: `omp_get_num_threads()` returns the number of threads in the **innermost** parallel region from which it is called. If called from a serial region, it returns 1. To query the potential team size before entering a parallel region, use `omp_get_max_threads()`.

0.1.4.3 4.3 Environment Variables Environment variables, set by the user before program execution, determine the behavior of the OpenMP runtime:

```
export OMP_NUM_THREADS=8           # Default number of threads
export OMP_STACKSIZE=8M           # Thread stack size
export OMP_SCHEDULE="dynamic,4"   # Default loop schedule
export OMP_PROC_BIND=spread       # Thread binding policy
export OMP_PLACES=cores           # Where threads can run
```

0.1.4.4 4.4 Conditional Compilation When the compiler processes OpenMP directives (enabled via flags such as `-fopenmp` for GCC/ICC or `-mp` for PGI), it defines the macro `_OPENMP`. This allows conditional compilation:

```
#ifdef _OPENMP
    my_thread_id = omp_get_thread_num();
#endif
```

This mechanism is essential for writing code that works correctly both with and without OpenMP, aiding in **correctness verification** and **portability** — particularly important in hybrid codes (e.g., MPI + OpenMP).

0.1.5 5. Static and Dynamic Extent

0.1.5.1 5.1 Static Extent The **static extent** of an OpenMP parallel region is defined by the lexical scope of the structured block — the code enclosed between the opening `{` and closing `}` braces of a `#pragma omp parallel` directive.

0.1.5.2 5.2 Dynamic Extent The **dynamic extent** extends beyond the static extent to include all function calls made from within the parallel region. If a function `foo()` is called from inside a parallel region, the code inside `foo()` is part of the dynamic extent and is subject to the same parallel execution semantics.

```
#pragma omp parallel
{
    double *array;
    int N;
    sum = foo(array, N);    // foo() is in the dynamic extent
}

double foo(double *A, int N)
{
    double mysum = 0;
    #pragma omp parallel for reduction(+:mysum) // "orphan" directive
```

```

    for (int ii = 0; ii < N; ii++)
        mysum += A[ii];
    return mysum;
}

```

Functions called within the dynamic extent can contain additional OpenMP directives — these are called **orphan directives** because they appear outside the lexical scope of any parallel construct, yet they execute within the dynamic extent of one.

0.1.6 6. Parallel Regions

0.1.6.1 6.1 Creating a Parallel Region The core construct in OpenMP is the **parallel region**:

```

#pragma omp parallel
{
    // Every thread in the team executes this block
}

```

When execution reaches the `#pragma omp parallel` directive, the runtime forks a pool of threads (the “team”). Every thread — including thread 0, which is the continuation of the original master thread — executes the code block. At the closing brace, an **implicit barrier** synchronizes all threads, and only thread 0 continues into the subsequent serial region.

A parallel region can be as short as a single line (without braces):

```

#pragma omp parallel
printf("hello world\n");

```

Other constructs that create or operate within parallel regions include:

Construct	Description
<code>#pragma omp parallel</code>	General parallel region
<code>#pragma omp parallel for</code>	Parallel region + loop work-sharing
<code>#pragma omp sections</code>	Work-sharing across code sections
<code>#pragma omp task</code>	Task-based parallelism

Note: `sections` and `task` are not parallel regions *per se* — they are work-sharing and tasking constructs that operate from **inside** a parallel region.

0.1.6.2 6.2 Clauses of the Parallel Construct The `parallel` directive supports several clauses:

Conditional creation (`if`):

```
#pragma omp parallel if(amount_of_work > threshold)
```

If the expression evaluates to zero (false), no parallel region is created — only thread 0 executes the block serially. This is important because the ~10 s overhead of creating a parallel region may exceed the benefit of parallelism for small workloads.

Thread count (`num_threads`):

```
#pragma omp parallel num_threads(n)
```

Opens the region with exactly n threads, overriding the default.

Processor binding (`proc_bind`):

```
#pragma omp parallel proc_bind(close) // or: spread, master
```

Controls the placement of threads onto hardware resources (discussed in detail in the NUMA awareness section).

0.1.7 7. The Thread Factory: Stack Layout and Memory Organization

0.1.7.1 7.1 Thread Stack Separation When a parallel region is opened, each thread receives its own **stack** — a separate region of the virtual address space with its own base pointer (RBP) and stack pointer (RSP). This can be verified experimentally: a variable declared outside the parallel region has the **same address** for all threads (shared), while a variable declared inside has a **different address** per thread (private, residing in each thread's stack).

0.1.7.2 7.2 Virtual Memory Layout of a Multi-Threaded Process The virtual address space of an OpenMP process during a parallel region is organized as:

```
[low addresses]
  code segment
  data segments (globals, BSS)
  heap (grows upward)
  ...
  thread N stack   (private variables of thread N)
  ...
  thread 2 stack   (private variables of thread 2)
  thread 1 stack   (private variables of thread 1)
  thread 0 stack   original process stack
  kernel space
[high addresses]
```


Each thread stack occupies a reserved region whose size equals the **maximum stack size**. The stacks are placed at regular intervals in the virtual address space — typically about 4 MB apart with GCC, or ~132 MB with Clang.

0.1.7.3 7.3 Stack Size Control The default thread stack size is implementation-dependent but can be configured via the environment variable:

```
export OMP_STACKSIZE=N
```

where N is followed by a suffix: B (bytes), K (kilobytes), M (megabytes), or G (gigabytes). Without a suffix, the value is interpreted as kilobytes.

Practical warnings:

- If the stack reservation is too **large** and many threads are created, the process may exhaust its virtual address space.
- If the stack reservation is too **small**, threads performing deep recursion or allocating large local arrays will trigger a **segmentation fault** (stack overflow).

0.1.7.4 7.4 Thread Count Limits The maximum number of threads is system-dependent. On Linux, it is bounded by:

$$n_{\max} \approx \frac{\text{Physical Memory}}{\text{Per-thread overhead}} \times \text{safety factor}$$

The OS limit can be checked with `cat /proc/sys/kernel/threads-max`. Within this limit, the OpenMP environment variable `OMP_THREAD_LIMIT` can further restrict the count. From code, `omp_get_max_threads()` returns the upper bound for the next team size.

0.1.8 8. Controlling the Number of Threads

By default, OpenMP spawns as many threads as there are available hardware threads. This can be overridden at three levels, listed in **increasing order of precedence**:

1. **Environment variable:** `export OMP_NUM_THREADS=4`
2. **Runtime function:** `omp_set_num_threads(4);`
3. **Directive clause:** `#pragma omp parallel num_threads(4)`

The OMP_DYNAMIC subtlety: if `OMP_DYNAMIC` is set to `TRUE` (or `omp_set_dynamic(1)` has been called), the runtime may use **fewer** threads than requested if it deems appropriate. When `OMP_DYNAMIC` is `FALSE` (the default on most implementations), the number requested is the number obtained.

0.1.9 9. Specializing Execution Within a Parallel Region

Not all code within a parallel region must be executed by every thread. OpenMP provides several constructs for restricting which threads execute specific code blocks.

0.1.9.1 9.1 The `single` Construct

```
#pragma omp single
{
    // Only one (arbitrary) thread executes this
}
```

One thread from the team executes the block; which one is implementation-defined. The other threads **wait at an implicit barrier** at the end of the block. A **memory flush** for all shared variables in scope is performed at the barrier, ensuring any modifications are visible to all threads.

This is the correct way to perform inherently serial work inside a parallel region:

```
int nthreads;
#pragma omp parallel
{
    #pragma omp single
    nthreads = omp_get_num_threads();
    // All threads see the correct value thanks to the implicit barrier + flush
}
```

0.1.9.2 9.2 The `masked` Construct

```
#pragma omp masked [filter(expression)]
{
    // Only the filtered thread(s) execute this
}
```

The `masked` construct is the **deterministic cousin** of `single`: instead of letting the runtime choose which thread executes, the programmer selects thread(s) explicitly via the `filter` clause. Without `filter`, it defaults to `filter(0)`, meaning only thread 0 executes.

The `filter` expression is evaluated by each thread; a thread executes the block **if and only if** the expression equals its thread ID:

```
// All threads execute (each sees filter == its own id):
#pragma omp masked filter(omp_get_thread_num())
{ ... }

// Only even-numbered threads execute:
int tid = omp_get_thread_num();
#pragma omp masked filter((tid % 2 == 0) ? tid : -1)
{ ... }
```

Critical difference from `single`: `masked` has **no implicit barrier** and **no implicit memory flush** at the end. If a shared variable is modified inside a `masked` block, other threads are not guaranteed to see the updated value unless an explicit `#pragma omp barrier` follows.

The `master` directive (which selected thread 0 exclusively) is now **deprecated** in OpenMP 5.1 in favor of `masked`.

0.1.9.3 9.3 `single` vs. `masked`: When to Use Which

Criterion	<code>single</code>	<code>masked</code>
Thread selection	Runtime-chosen (arbitrary)	Deterministic (via <code>filter</code>)
Implicit barrier	Yes	No
Memory flush	Yes	No
Overhead	Higher (synchronization infrastructure)	Lower (only thread ID test)
Use case	Result must be visible to all threads immediately	Assigning work to specific threads; no waiting needed

0.1.9.4 9.4 The `critical` Construct

```
#pragma omp critical
{
    // Only one thread at a time executes this
}
```

The `critical` construct implements **mutual exclusion**: all threads will eventually execute the block, but **only one at a time**. At the entry point, a lock is acquired with a memory flush of all shared variables; there is **no synchronization at exit** — threads leave as soon as they finish.

This is a **serialization point**: if the work inside the critical section is comparable to the work outside it, the program is effectively serialized.

Named critical regions: by default, OpenMP maintains a single global critical section. All unnamed critical regions are considered part of the **same lock**. This means two unrelated critical regions will block each other:

```
#pragma omp critical
{ /* region A */ }    // A thread here blocks entry to region B
#pragma omp critical
{ /* region B */ }
```

To avoid this performance trap, use **named** critical regions:

```
#pragma omp critical(update_counter)
{ counter++; }
#pragma omp critical(log_message)
{ fprintf(logfile, ...); }
```

Named regions have **independent locks** and can be entered simultaneously by different threads.

0.1.9.5 9.5 The atomic Construct

```
#pragma omp atomic [read | write | update | capture]
x binop= expr;
```

The **atomic** construct protects a **single memory location** using hardware-level atomic instructions (such as `LOCK ADD` on x86). It is significantly cheaper than **critical** — there is no software mutex, no thread queuing, just a cache-line lock for the duration of the instruction.

However, **atomic** is limited to specific forms of assignment on scalar variables:

Clause	Form	Effect
<code>read</code>	<code>var = mem</code>	Atomic read of <code>mem</code>
<code>write</code>	<code>mem = var</code>	Atomic write to <code>mem</code>
<code>update</code> (default)	<code>mem++, mem binop= expr</code>	Atomic read-modify-write
<code>capture</code>	<code>val = mem; mem binop= expr</code>	Atomic update + capture old/new value

Fundamental distinction: **atomic** and **critical** protect **different things** and are **not mutually exclusive**:

- **critical** protects a **code region** (software mutex)
- **atomic** protects a **memory location** (hardware instruction)

A thread executing a **critical** region that modifies variable `x` will **not** block another thread performing an **atomic** on the same `x`. Using both mechanisms inconsistently on the same variable can lead to data races.

0.1.9.6 9.6 The barrier Construct

```
#pragma omp barrier
```

A **barrier** is the most forceful synchronization tool: all threads halt and wait until every thread in the team has arrived. Upon departure from the barrier, it is guaranteed that all other threads have at least reached it. A **memory flush** of all shared variables in scope occurs at the barrier.

Use barriers sparingly — they are expensive (all threads idle waiting for the slowest one) and can cause deadlocks if placed inside conditional branches that not all threads reach (which is illegal in OpenMP but may not be caught by the compiler).

Implicit barriers exist at the end of **parallel** regions, **single** blocks, and worksharing constructs (unless suppressed with **nowait**).

0.1.10 10. Shared and Private Variables: The Scoping Rules

0.1.10.1 10.1 Default Rules Understanding what is **shared** and what is **private** is the single most important aspect of correct OpenMP programming. The default rules are:

Declaration location	Default scope	Rationale
Outside the parallel region	Shared	All threads see the same address
Inside the parallel region	Private	Each thread has its own copy in its stack
Global variables	Shared	Exception: threadprivate
Dynamically allocated memory	Shared (heap)	Pointer follows the rules above
Loop indices on <code>#pragma omp for</code>	Private	Automatically privatized

Precise definitions: - A variable is **shared** if its address (`&var`) is the **same** for all threads in the team. - A variable is **private** if its address is **different** for each thread.

0.1.10.2 10.2 The private Clause

```
int i, j, k;
double *array;

#pragma omp parallel private(i, k)
{
    // j and array are shared
    // i and k are private - unique to each thread, living in each thread's stack
    // They are UNINITIALIZED - not copies of the originals
}
```

The **private** clause creates **new** variables in each thread's stack with the same type and name as the originals, but at **different memory addresses** and with **undefined initial values**. The original variables in the serial region are untouched and inaccessible from within the parallel region.

Pointer subtlety: if a pointer is declared as **private**, only the pointer object itself (8 bytes on x86-64) is privatized. The pointed-to memory is **not** duplicated. The private copy is **uninitialized** — it does **not** inherit the value of the original pointer. Dereferencing it is undefined behavior:

```
double *array = malloc(N * sizeof(double));
#pragma omp parallel private(array)
{
    // array is private but UNINITIALIZED - does NOT point to the malloc-ed memory!
    array[0] = 42.0; // UNDEFINED BEHAVIOR
}
```

Best practice advice: avoid the **private** clause for scalars. Instead, declare private variables directly inside the parallel region. This eliminates the confusing situation where the same name refers to different memory locations depending on scope:

```

int i;
// ... using i in serial code ...

#pragma omp parallel
{
    int j; // Private by declaration - clear and unambiguous
           // i is still accessible here and is shared
}

```

0.1.10.3 10.3 The default(none) Clause

```

#pragma omp parallel default(none) private(i, k) shared(j, array)

```

With `default(none)`, the compiler issues an error if any variable in scope is not explicitly listed in a `shared()` or `private()` clause. This acts as a safety net against accidental scoping errors, particularly in large codebases with many variables.

0.1.10.4 10.4 The firstprivate Clause

```

double *array = malloc(N * sizeof(double));

#pragma omp parallel firstprivate(array)
{
    // array is private, BUT initialized to the value of the original pointer
    // It still points to the malloc-ed memory
    array[omp_get_thread_num()] = 42.0; // Valid
}

```

`firstprivate` creates a private copy for each thread, **initialized** with the value the variable has at the point of entering the parallel region. For scalars, the value is copied; for pointers, the **pointer value** (the address it points to) is copied — not the pointed-to data.

0.1.10.5 10.5 The lastprivate Clause

```

double last_result;
#pragma omp parallel for lastprivate(last_result)
for (int j = 0; j < N; j++)
    last_result = compute(j);
// last_result == compute(N-1), the value from the last iteration

```

`lastprivate` applies to `for`, `sections`, and `taskloop` constructs. Each thread has a private copy (uninitialized, like `private`), but after the construct completes, the value from the **last iteration** (highest index) is copied back to the original variable in the serial scope.

0.1.10.6 10.6 The `threadprivate` Directive

```
int myN;
double *array;
#pragma omp threadprivate(myN, array)

#pragma omp parallel
{
    array = malloc(...);
    myN = omp_get_thread_num();
}
// ... serial region ...
#pragma omp parallel
{
    // array still points to the previously allocated memory
    // myN still has the previously assigned value
}
```

`threadprivate` applies to **global** (or file-scope static) variables and makes them private across the **entire program lifetime**, not just within a single parallel region. Each thread gets its own persistent copy that survives between parallel regions.

Restriction: when using `threadprivate`, dynamic thread creation is not allowed — the number of threads must remain constant across all parallel regions.

0.1.10.7 10.7 The `copyin` Clause

```
int golden_values[3];
#pragma omp threadprivate(golden_values)

get_golden_values(); // Fills thread 0's copy
#pragma omp parallel copyin(golden_values)
{
    // Each thread's golden_values is now a copy of thread 0's
}
```

`copyin` provides a **broadcast** mechanism for `threadprivate` variables: at the start of the parallel region, the value from thread 0 is copied into every other thread's private copy.

0.1.10.8 10.8 The `copyprivate` Clause

```
#pragma omp parallel
{
    double seed[2];
    #pragma omp single copyprivate(seed)
    {
        seed[0] = compute_seed_0();
    }
}
```

```

    seed[1] = compute_seed_1();
}
// All threads now have a copy of seed
}

```

`copyprivate` applies only to `single` constructs. After the `single` block executes, the private variables listed are **propagated** from the executing thread to all others. The copy completes before any thread leaves the implicit barrier at the end of `single`.

0.1.10.9 10.9 Summary of Data-Sharing Clauses

Clause	Scope	Initialization	Persistence	Applies to
<code>shared</code>	Same address for all	N/A	N/A	<code>parallel</code>
<code>private</code>	Per-thread copy	Undefined	Dies with region	<code>parallel</code> , <code>for</code>
<code>firstprivate</code>	Per-thread copy	Copied from original	Dies with region	<code>parallel</code> , <code>for</code>
<code>lastprivate</code>	Per-thread copy	Undefined	Last iteration → original	<code>for</code> , <code>sections</code>
<code>threadprivate</code>	Per-thread global	Persistent	Across regions	Global variables
<code>copyin</code>	Broadcast	From thread 0	N/A	<code>threadprivate</code> vars
<code>copyprivate</code>	Broadcast	From <code>single</code> thread	N/A	<code>single</code> block

0.1.11 11. Nested Parallelism

OpenMP explicitly permits **nested** parallel regions:

```

#pragma omp parallel num_threads(4)
{
    // Level 0: 4 threads
    #pragma omp parallel num_threads(2)
    {
        // Level 1: each of the 4 threads spawns 2 more → up to 8 threads
    }
}

```

Whether nesting is active depends on environment variables:

- `OMP_MAX_ACTIVE_LEVELS=n` — maximum depth of active nesting (default usually 1, meaning inner regions are serialized)
- `OMP_NUM_THREADS=N0,N1,...` — comma-separated list giving team size at each nesting level

The older `OMP_NESTED=TRUE|FALSE` is **deprecated** since OpenMP 5.0 but still supported by most compilers.

Practical caution: nested parallelism is rarely beneficial. The overhead of creating nested teams is non-trivial, and there is a risk of **oversubscription** (e.g., 4 outer $\times$ 4 inner = 16 threads on an 8-core machine $\rightarrow$ contention and context switching).

0.1.12 12. Parallel Loops

0.1.12.1 12.1 Motivation Loops are one of the most common computational structures in HPC. A vast proportion of compute-intensive code resides in loops. Until OpenMP version 2.x, the standard was essentially about quickly and effectively parallelizing **for** loops.

0.1.12.2 12.2 The `parallel for` Construct The fundamental loop-parallelization directive distributes iterations across threads:

```
#pragma omp parallel for
for (int i = 0; i < N; i++)
    a[i] = compute(i);
```

The iteration space $\{0, 1, \dots, N - 1\}$ is partitioned among threads, with each thread processing a subset of iterations. The loop index `i` is automatically **private**.

Important distinction between `omp for` inside an existing parallel region and `omp parallel for`:

// Case A: for inside parallel - single loop shared among existing threads

```
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0; i < N; i++)
        work(i);
}
```

// Case B: parallel for - each thread spawns a NEW parallel region

```
#pragma omp parallel
{
    #pragma omp parallel for // NESTED parallelism!
    for (int i = 0; i < N; i++)
        work(i);
}
```

In Case A, the single **for** loop is subdivided among threads already in the pool. In Case B, **each** thread in the outer region creates a new parallel region, and within each new region the threads share the **for** cycle. If nesting is disabled, each thread executes the entire loop sequentially.

0.1.12.3 12.3 Loop Scheduling Policies The `schedule` clause controls how iterations are distributed:

```
#pragma omp parallel for schedule(scheduling_type [, chunk_size])
```

Policy	Description	Best for
<code>static [, chunk]</code>	Iterations divided into chunks distributed in round-robin order	Uniform workload per iteration
<code>dynamic [, chunk]</code>	Chunks distributed on a first-arrived-first-served basis	Unpredictable workload per iteration
<code>guided [, chunk]</code>	Like <code>dynamic</code> , but chunk size decreases proportionally to remaining iterations	Decreasing workload variance
<code>runtime</code>	Policy set via <code>OMP_SCHEDULE</code> env variable	Deferred decision

Static scheduling: chunks are assigned with a rigid round-robin sequence. The assignment is purely arithmetic — no runtime overhead. Each thread knows exactly which iterations it owns.

Dynamic scheduling: chunks are assigned to the first thread that requests work. This involves a **queue** with associated overhead, but provides better load balancing when iteration costs vary.

Rule of thumb: static scheduling is most effective when each iteration brings the same computational work (predictable load), because the direct assignment has minimal overhead. Dynamic scheduling is most effective when computational load per iteration is unpredictable.

0.1.12.4 12.4 Clauses in `parallel for`

```
#pragma omp for
    schedule(policy [, chunk])
    ordered
    private(var_list)
    firstprivate(var_list)
    lastprivate(var_list)
    shared(var_list)
    reduction(op: var_list)
    collapse(n)
    nowait
```

reduction(op: var_list): the runtime creates a **private copy** of each listed variable for each thread (initialized to the identity element of the operator: 0 for +, 1 for *, type min/max for max/min), then **combines** all copies at the end using the specified operator.

Possible operators: +, *, -, max, min, &, &&, |, ||

```
#pragma omp parallel for reduction(+: sum)
for (int i = 0; i < N; i++)
    sum += a[i];
```

collapse(n): enables parallelization of multiple **perfectly nested** loops:

```
#pragma omp for collapse(2)
for (int i = 0; i < Nrows; i++)
    for (int j = 0; j < Ncols; j++)
        A[i][j] = B[i][j] * C[i][j];
```

The total iteration space becomes $N_{\text{rows}} \times N_{\text{cols}}$, distributed across threads. The loops must be **perfectly nested** — no statements between the **for** headers.

nowait: suppresses the implicit barrier at the end of the worksharing construct:

```
#pragma omp for nowait
for (int i = 0; i < N; i++)
    a[i] = compute_a(i);

#pragma omp for
for (int i = 0; i < M; i++)
    b[i] = compute_b(i); // b does NOT depend on a - safe with nowait
```

0.1.13 13. Data Races and Their Anatomy

0.1.13.1 13.1 Definition A **data race** occurs when at least two memory accesses:

1. Target the **same** memory location
2. Are performed **concurrently** by different threads
3. Are not synchronization operations
4. At least one is a **write**

A **race condition** is a semantic error in the code: due to the random ordering of events, it leads to non-deterministic behavior and incorrect results.

0.1.13.2 13.2 Detailed Example Consider the unsynchronized loop:

```
#pragma omp parallel for
for (int i = 0; i < N; i++)
    sum += a[i]; // DATA RACE on sum
```

The single-line `sum += a[i]` translates to multiple machine instructions:

```

mov  sum → regA      ; fetch sum from memory into register A
add  a[i], regA       ; add a[i] to register A
mov  regA → sum       ; store result back to memory

```

With 2 threads and $a[] = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ (static schedule: thread 0 processes $a[0:4]$, thread 1 processes $a[5:9]$), interleaved execution can cause one thread's update to overwrite another's:

Cycle	sum in memory	Thread 0	Thread 1
0	0	—	fetch $\text{sum} \rightarrow \text{regA}$ ($\text{regA}=0$)
1	0	fetch $\text{sum} \rightarrow \text{regA}$ ($\text{regA}=0$)	add $a[5]$ ($\text{regA}=5$)
2	0	add $a[1]$ ($\text{regA}=1$)	store $\text{regA} \rightarrow \text{sum}$
3	5	store $\text{regA} \rightarrow \text{sum}$	...
4	1	...	...

Thread 1's contribution (5) is **lost** — overwritten by thread 0's stale value. The result will be smaller than the correct sum.

0.1.14 14. False Sharing

0.1.14.1 14.1 Definition **False sharing** occurs when threads explicitly access **different** memory locations that happen to be mapped onto the **same cache line**. Although no two threads touch the same variable, the cache coherence protocol treats the entire cache line as a unit — any write to one element invalidates the line in all other cores' caches.

This is distinct from intentional memory sharing (which is the basis of shared-memory programming). False sharing causes severe performance degradation when it occurs in tight loops.

0.1.14.2 14.2 Mechanism (MESI Protocol) Consider 2 threads writing to adjacent elements of a shared array:

1. **Step 0:** both threads read their target location → the entire cache line is loaded into both cores' L1 caches, flagged as **Shared (S)**.
2. **Step 1:** thread 0 writes its element → the line is flagged as **Modified (M)** in core 0 and **Invalid (I)** in core 1.
3. **Step 2:** thread 1 wants to write its element → the line must be **re-fetched** from core 0 (cache-to-cache transfer, 30–100 cycles) because it is flagged as Invalid.
4. **Step 3:** thread 1 writes → the line is now **M** in core 1 and **I** in core 0.
5. The cycle repeats on every write, causing continuous **cache-line bouncing**.

0.1.14.3 14.3 Example: False Sharing in Reduction

```
double sum[nthreads]; // Elements are contiguous + same cache line!
#pragma omp parallel
{
    int me = omp_get_thread_num();
    #pragma omp for
    for (int i = 0; i < N; i++)
        sum[me] += a[i]; // FALSE SHARING
}
```

With 8 threads, the array `sum[8]` occupies 64 bytes — exactly **one cache line**. Every thread writes to a different element, but every write invalidates the entire line for all other threads.

Naive fix (padding to cache-line boundaries):

```
double sum[nthreads * 8]; // Pad by factor of 8 (64 bytes / 8 bytes per double)
sum[me * 8] += a[i];
```

This works but wastes memory and hard-codes a magic number (not portable).

Correct fix (local accumulator):

```
#pragma omp parallel
{
    double local_sum = 0; // On thread's stack - no sharing
    #pragma omp for
    for (int i = 0; i < N; i++)
        local_sum += a[i];
    #pragma omp atomic
    sum += local_sum; // One synchronization per thread
}
```

Or equivalently, use the `reduction` clause.

0.1.15 15. Solving the Reduction Problem

0.1.15.1 15.1 The Hierarchy of Solutions The parallel reduction problem illustrates a fundamental principle. Consider computing $\text{sum} = \sum_{i=0}^{N-1} a[i]$:

Approach 1: critical on every iteration — worst performance:

```
#pragma omp parallel for
for (int i = 0; i < N; i++)
    #pragma omp critical
    sum += a[i];
```

Each iteration enters a mutex, causing complete serialization plus lock overhead.

Approach 2: atomic on every iteration — better but still poor:

```
#pragma omp parallel for
for (int i = 0; i < N; i++)
    #pragma omp atomic
    sum += a[i];
```

Hardware atomic instructions are cheaper than mutexes (~30–80 cycles vs. ~200+ cycles), but contention still grows with thread count.

Approach 3: Local accumulator + single final atomic — near-optimal:

```
#pragma omp parallel
{
    double local_sum = 0;
    #pragma omp for
    for (int i = 0; i < N; i++)
        local_sum += a[i];
    #pragma omp atomic
    sum += local_sum;
}
```

Synchronization is pushed to the **boundary**: N iterations with no synchronization, one **atomic** at the end.

Approach 4: reduction clause — syntactic sugar for Approach 3:

```
#pragma omp parallel for reduction(+: sum)
for (int i = 0; i < N; i++)
    sum += a[i];
```

The runtime automatically creates per-thread private copies (initialized to 0) and combines them at the end.

0.1.15.2 15.2 Quantitative Comparison

Approach	Sync operations	Cost per sync	Total sync cost	Scalability
critical per iteration	$N \times T$	~200+ cycles	$N \times T \times 200$	Anti-scales
atomic per iteration	$N \times T$	~30–80 cycles	$N \times T \times 50$	Poor
Local + final atomic	T	~30–80 cycles	$T \times 50$	Near-perfect
reduction	T	~30–80 cycles	$T \times 50$	Near-perfect

The fundamental lesson: do not synchronize inside a tight loop if it can be avoided. Accumulate locally, combine once. This is the most common performance pattern in shared-memory programming.

0.1.16 16. Performance Case Study: Monte Carlo Estimation of

0.1.16.1 16.1 The Algorithm Consider the unit square $[0, 1] \times [0, 1]$ and the quarter-circle of radius 1 inscribed in it. The area of the quarter-circle is $\pi/4$. Generating N random points uniformly in the square, the fraction landing inside the quarter-circle converges to $\pi/4$ as $N \rightarrow \infty$:

$$\pi \approx 4 \times \frac{N_{\text{valid}}}{N_{\text{total}}}$$

where N_{valid} is the count of points satisfying $x^2 + y^2 < 1$.

The statistical error decreases as $1/\sqrt{N}$, making this impractical for computing π but ideal as a **weak scaling** benchmark: each thread generates N points (total work scales linearly with thread count), so perfect parallelization yields **constant** wall-clock time.

0.1.16.2 16.2 Thread-Safe Random Number Generation A critical prerequisite is **thread-safe** pseudo-random number generation. The standard `drand48()` function maintains its state in a **single global buffer** — a static variable inside the C library. When multiple threads call it concurrently:

1. **Data race:** interleaved read-modify-write operations corrupt the linear congruential sequence.
2. **Statistical bias:** the corrupted sequence can exhibit correlations, reduced period, or other defects, causing the π estimate to converge to the **wrong value**.

Solution: use the reentrant `erand48()`, which takes a pointer to a caller-supplied **unsigned short[3]** state array. Each thread maintains its own seed on its own stack — no shared mutable state:

```
unsigned short myseeds[3] = {(short)(SEED+(myid)),
                             (short)(SEED^(myid*3+1)),
                             (short)(SEED&(myid*4+2))};

seed48(myseeds);
double x = erand48(myseeds);
```

General principle: any library function storing mutable state internally is a trap in multi-threaded contexts. Always check for reentrant variants (POSIX `_r` suffix, C11 `_s` suffix).

0.1.16.3 16.3 Impact of Synchronization Strategy The following table summarizes results from a 4-socket Intel Xeon Gold 5118 (48 cores) at $T = 48$ threads:

Version	Strategy	Normalized time
v0	critical every iteration	$\sim 2600\times$ baseline
v0b	critical only on hits ($\sim 78.5\%$)	$\sim 2000\times$ baseline
v1	atomic every iteration	$\sim 160\times$ baseline
v2	Local accumulator + final atomic	$\sim 1\times$ baseline

0.1.16.4 16.4 Topology Dependence The same benchmark on two architectures with comparable clock speeds (2.3 GHz) but different topologies reveals dramatic differences:

2-socket Broadwell-EP (E5-2697 v4): any core reaches any other core’s cache in at most **1 inter-socket hop** (direct QPI link). At 36 threads: v0 $600\times$, v1 $60\times$, v2 $1\times$.

4-socket Skylake-SP (Gold 5118): sockets form a **ring** topology with only 2 UPI links per socket. Socket 0 to Socket 2 requires **2 hops**. At 36 threads: v0 $1500\times$, v1 $100\times$, v2 $1\times$.

The contention-heavy versions are **2–3 $\times$ worse** on the 4-socket system because:

1. **Longer coherence paths:** 2-hop maximum vs. 1-hop maximum.
2. **More coherence participants:** each write must potentially invalidate copies in 3 remote sockets vs. 1.
3. **UPI link congestion:** intermediate sockets carry transit traffic on top of their own.

The synchronization-free version (v2) produces **identical** performance on both platforms — the topology is irrelevant when there is no coherence traffic during the hot loop.

Takeaway: the cost of synchronization is **not** a fixed number. Same code, same compiler, same clock — different topology yields different performance. The only robust strategy is to **eliminate unnecessary synchronization altogether**.

0.1.17 17. NUMA Awareness and Thread Affinity

0.1.17.1 17.1 The NUMA Problem Modern multi-socket systems employ **Non-Uniform Memory Access (NUMA)** architectures: memory is physically partitioned into banks, each connected to a specific socket. All sockets are interconnected so every core can access all memory, and a cache-coherence protocol maintains data consistency. The aggregate bandwidth scales with the number of sockets.

However, **access time is no longer uniform:** accessing memory on the local bank is significantly faster than accessing a remote bank. This has severe consequences for code design and execution.

0.1.17.2 17.2 The Three Questions: Where? Who? What? Minimizing remote memory accesses depends on three factors:

1. **Where:** in which physical memory bank do the data reside?
2. **Who:** which thread accesses the data, and on which core does it run?
3. **What:** how is the workload distributed among threads?

The goal is to minimize **remote memory accesses** by ensuring that each thread's data reside in the memory bank closest to the core on which it runs.

0.1.17.3 17.3 Thread Affinity: Places and Binding OpenMP provides two fundamental concepts for controlling thread placement:

Places define **where** threads can run — the set of hardware resources available:

Place type	Meaning
threads	Each place = one hardware thread (strand)
cores	Each place = one physical core (may have multiple strands)
sockets	Each place = one physical socket (with all its cores)

Set via: `export OMP_PLACES={sockets | cores | threads}`

Binding defines **how** software threads are mapped onto places:

Policy	Description
false	No policy; OS decides placement and may migrate threads
true	No specific policy, but OS forbidden from migrating threads
close	Threads placed on consecutive places, as close as possible
spread	Threads distributed as evenly as possible across places
master	All threads placed on the same place as the master thread

Set via: `export OMP_PROC_BIND={false | true | close | spread | master}`

Or per-region: `#pragma omp parallel proc_bind(spread)`

0.1.17.4 17.4 Close vs. Spread Binding **Close binding** assigns threads to **consecutive** places, starting from the master thread's place:

- When $T \leq P$ (threads $\leq$ places): each thread gets a unique place, assigned consecutively by thread ID.
- When $T > P$: threads are split into P subsets, each assigned to one place, with $|S_i| \leq \lceil T/P \rceil$.

Spread binding distributes threads as **evenly as possible** across all available places:

- When $T \leq P$: the place list is divided into T subpartitions of $\lfloor P/T \rfloor$ to $\lceil P/T \rceil$ consecutive places each. One thread is assigned per subpartition.
- When $T > P$: the place list is divided into P single-place subpartitions, each hosting $\lfloor T/P \rfloor$ to $\lceil T/P \rceil$ threads.

When close and spread differ: consider 2 threads on 4 sockets with `OMP_PLACES=sockets`:

- **Close:** threads on sockets 0 and 1 (consecutive)
- **Spread:** threads on sockets 0 and 2 (maximally spread)

0.1.17.5 17.5 Optimal Placement Depends on the Algorithm

Scenario	Preferred binding	Rationale
Threads share data frequently	close	Share L2/L3 cache, minimize synchronization latency
Threads work on independent memory	spread	Maximize aggregate bandwidth, minimize contention
Hybrid MPI+OpenMP	spread at MPI level, close at OpenMP level	Distribute across sockets, cluster within socket

0.1.17.6 17.6 Advanced Place Definitions Places can be defined with explicit hardware thread IDs using interval notation:

```
export OMP_PLACES="{0}:4:12"           # → {0}, {12}, {24}, {36}
export OMP_PLACES="{0:4:1}:4:12"       # → {0,1,2,3}, {12,13,14,15}, ...
export OMP_PLACES="{0:12}:4:12"        # Equivalent to "sockets" on a 4-socket, 12-core system
```

The syntax `{start:count:stride}` generates the series:

start, start + stride, start + 2 × stride, ..., start + (count − 1) × stride

The `!` operator excludes intervals. Places are **static** — they cannot be changed at runtime.

0.1.18 18. Memory Allocation on NUMA Systems

0.1.18.1 18.1 The Touch-First Policy Most operating systems implement a **touch-first** (or first-touch) memory allocation policy: physical memory pages are allocated in the memory bank **closest to the core** that first accesses (“touches”) the data.

Consequence: if a single thread initializes all data in a serial region, all pages are allocated in that thread’s local memory bank. When the parallel region begins, all other threads must perform **remote memory accesses**, degrading bandwidth.

```
// BAD: all data allocated on thread 0's memory bank
double *a = (double*)calloc(N, sizeof(double)); // calloc touches all pages
for (int i = 0; i < N; i++)
    a[i] = initialize(i);

#pragma omp parallel for reduction(+: sum)
for (int i = 0; i < N; i++)
    sum += a[i]; // Remote access for all threads except thread 0
```

0.1.18.2 18.2 The Touch-By-All Policy To achieve optimal data placement, each thread should **first touch** the data it will subsequently operate on:

```
// GOOD: each thread touches its own data
double *a = (double*)malloc(N * sizeof(double)); // malloc does NOT touch pages

#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++)
    a[i] = initialize(i); // Each thread touches its portion

#pragma omp parallel for schedule(static) reduction(+: sum)
for (int i = 0; i < N; i++)
    sum += a[i]; // Local access for all threads (same schedule!)
```

Critical detail: the initialization loop and the computation loop must use the **same scheduling** so that each thread operates on the pages it allocated.

0.1.18.3 18.3 malloc vs. calloc in NUMA Context

Function	Physical mapping	Initialization
malloc	Pages mapped on first touch (lazy)	None — contents undefined
calloc	All pages mapped immediately (zero-fill touches everything)	All bytes set to zero

Using `calloc` in the serial region causes **all** pages to be allocated on thread 0's memory bank (because `calloc` touches every page during zero-initialization). Using `malloc` defers physical mapping until the first actual access, allowing the touch-by-all pattern to control placement.

0.1.18.4 18.4 Topology Discovery Tools Several utilities are available for inspecting the hardware topology of an HPC node:

Tool	Usage
<code>numactl --hardware</code>	Summary of NUMA topology
<code>lscpu</code>	CPU configuration details

Tool	Usage
<code>lstopo -s</code>	Graphical topology summary (from <code>hwloc</code>)
<code>likwid-topology [-g]</code>	Detailed topology with cache information
<code>cat /proc/cpuinfo</code>	Raw CPU information
<code>taskset</code>	Set/retrieve CPU affinity for processes
<code>numactl --cpunodebind / --membind</code>	Bind execution/memory to specific NUMA nodes

0.1.19 19. The OpenMP Memory Model

0.1.19.1 19.1 Why a Memory Model Is Needed In a shared-memory system, all threads see a common address space. However, modern CPUs have:

- **Private L1/L2 caches** per core
- **Store buffers** that delay writes
- **Out-of-order execution** that reorders loads and stores

While coherence protocols ensure **eventual** consistency, “eventual” can mean hundreds or thousands of CPU cycles. The OpenMP memory model defines:

1. **When** writes become visible to other threads
2. **What ordering guarantees** exist at synchronization points
3. **How** to force visibility with flush operations

The memory model is a **contract** between the programmer and the implementation: code that abides by the rules will behave correctly on any compliant implementation. Code that violates the rules — e.g., by reading a shared variable without proper synchronization — exhibits **undefined behavior**.

0.1.19.2 19.2 Temporary View vs. Shared Memory OpenMP introduces a conceptual distinction:

- **Shared memory:** the “ground truth” that all threads eventually agree on — the state of main memory if the entire system were frozen.
- **Temporary view:** what each thread currently “sees” — including values in CPU registers, store buffers, and private cache lines not yet synchronized.

The key insight is that a thread’s temporary view **may be stale**: it may contain values that another thread has already overwritten in shared memory, but that update has not propagated yet.

0.1.19.3 19.3 The Flush Operation A **flush** synchronizes a thread's temporary view with shared memory:

1. **Pushes** all pending writes from the thread's temporary view to shared memory (makes the calling thread's writes visible to others)
2. **Pulls** fresh values from shared memory for subsequent reads (makes others' writes visible to the calling thread)

Critical subtlety: a flush is a **point-wise** operation — it synchronizes **only** the calling thread's view. It does not wait for other threads. For communication to work, **both sides must flush**: one to push, one to pull.

0.1.19.4 19.4 Implicit Flush Points OpenMP inserts implicit flush operations at specific synchronization points:

Construct	Flush occurs...
#pragma omp barrier	At entry and exit
#pragma omp critical	At entry and exit
#pragma omp parallel	At entry (all threads); at exit (implicit barrier)
#pragma omp for (no nowait)	At exit (implicit barrier)
#pragma omp single (no nowait)	At exit (implicit barrier)
#pragma omp atomic	Only for the memory location being updated
Lock operations	omp_set_lock, omp_unset_lock, etc.

In most simple cases, implicit flushes at barriers suffice — explicit flush is unnecessary.

0.1.19.5 19.5 The Explicit Flush Directive

```
#pragma omp flush           // Flush all shared variables
#pragma omp flush(x, y)    // Flush only x and y (more efficient)
```

Explicit flush is needed in specific scenarios such as **spin-waiting** on a flag variable:

```
// INCORRECT - may spin forever
int flag = 0;
#pragma omp parallel num_threads(2)
{
    if (myid == producer) {
        do_work();
        flag = 1;           // Write may stay in store buffer
    } else {
        while (flag == 0)    // May read stale value forever
            ;
        use_result();
    }
}
```

```

}

// CORRECT - with explicit flush
int flag = 0;
#pragma omp parallel num_threads(2)
{
    if (myid == producer) {
        do_work();
        flag = 1;
        #pragma omp flush(flag) // Push to shared memory
    } else {
        while (flag == 0)
            #pragma omp flush(flag) // Pull from shared memory
            ;
        use_result();
    }
}

```

0.1.19.6 19.6 Flush Is Not...

- **Not a barrier:** it synchronizes the calling thread's view with shared memory but does not wait for other threads.
- **Not mutual exclusion:** for that, use `critical` or `locks`.
- **Not an atomic update:** for that, use `atomic`.
- **Not a task dependency mechanism:** for that, use the `depend` clause or `barriers`.

0.1.19.7 19.7 The `nowait` Clause and Its Perils By default, worksharing constructs have an **implicit barrier** at their end. The `nowait` clause removes this barrier:

```

// SAFE: loops are independent
#pragma omp for nowait
for (int i = 0; i < N; i++)
    a[i] = compute_a(i);

#pragma omp for nowait
for (int i = 0; i < M; i++)
    b[i] = compute_b(i); // b does NOT depend on a

// UNSAFE: second loop reads a[]
#pragma omp for nowait
for (int i = 0; i < N; i++)
    a[i] = compute_a(i);

#pragma omp for
for (int i = 0; i < N; i++)
    b[i] = a[i] * 2; // May read a[i] before it's written!

```

Rule: if the next computation reads data produced by the previous worksharing construct, the barrier **must not** be removed.

0.1.19.8 19.8 Architecture-Dependent Behavior The behavior of code with missing synchronization depends on the hardware’s memory ordering model:

- **x86-64 (TSO — Total Store Order):** stores are ordered, and if the consumer reloads from memory and sees a flag update, it is guaranteed to also see prior data updates. However, the **compiler** may hoist loads out of loops (optimizing `while(flag==0)` into a single register load), making hardware guarantees irrelevant.
- **ARM, POWER, RISC-V:** weaker memory models permit more aggressive reordering for performance. Code that works on x86-64 may fail on these architectures.

The OpenMP memory model abstracts over these hardware differences, but only if the programmer **respects the synchronization contract**. Relying on hardware-specific behavior produces non-portable code.

0.1.20 20. Internal Control Variables (ICVs)

OpenMP behavior is governed by **Internal Control Variables** — conceptual variables accessed via environment variables (before program start) or runtime functions (during execution).

ICVs have different scopes:

Scope	Description	Examples
Global	Applies to the whole program, fixed at startup	OMP_CANCELLATION
Device	Per-device (relevant for GPUs, co-processors)	OMP_STACKSIZE, OMP_WAIT_POLICY, OMP_SCHEDULE
Task	Local to each task; inherited by child tasks	OMP_NUM_THREADS, OMP_DYNAMIC, OMP_PROC_BIND

Key ICVs:

Environment Variable	Scope	Accessible from Code	Purpose
OMP_NUM_THREADS	Task	get/set	Default team size
OMP_MAX_ACTIVE_LEVELS	Task	get/set	Maximum nesting depth
OMP_DYNAMIC	Task	get/set	Allow runtime to adjust team size
OMP_STACKSIZE	Device	—	Thread stack reservation

Environment Variable	Scope	Accessible from Code	Purpose
OMP_THREAD_LIMIT	Task	get	Hard upper bound on threads
OMP_WAIT_POLICY	Device	—	Idle thread behavior (ACTIVE=spin, PASSIVE=sleep)
OMP_PROC_BIND	Task	set	Thread-to-core affinity
OMP_PLACES	Task	—	Where threads can run
OMP_SCHEDULE	Device	get/set	Default loop schedule

0.1.21 21. Summary of Practical Guidelines

1. **Declare private variables inside the parallel region.** Avoid the `private()` clause for scalars — it creates confusion. The exception is `firstprivate`, which has genuinely useful semantics.
2. **Always reason about the barrier.** Every time a shared variable is written, ask: when will other threads see this? `single` provides an implicit barrier; `masked` does not.
3. **Prefer atomic over critical** when possible. `atomic` uses hardware support and is orders of magnitude cheaper. For complex operations, `critical` (or algorithm restructuring) is necessary.
4. **Use named critical regions.** The global unnamed critical section is a performance trap — independent critical regions should not block each other.
5. **Minimize synchronization in tight loops.** Accumulate locally, reduce once. This is the single most impactful performance pattern.
6. **Remember the overhead.** Opening a parallel region costs ~10 s. Use the `if` clause to skip parallelism when the workload is too small.
7. **Respect the NUMA topology.** Use the touch-by-all pattern for initialization. Match the scheduling of initialization and computation loops. Use `OMP_PLACES` and `OMP_PROC_BIND` to control thread placement.
8. **Use thread-safe library functions.** Any function with internal mutable state (`drand48`, `strtok`, `ctime`) is unsafe in multi-threaded contexts. Use reentrant variants (`erand48`, `strtok_r`).
9. **Do not fight non-determinism.** Thread execution order is not guaranteed. Design algorithms so that correctness does not depend on execution order.
10. **Synchronization costs are not portable.** Same code, same clock, different topology → different performance for contention-heavy code. The only robust strategy is to eliminate unnecessary synchronization.